Prediction of Falcon-9 First-Stage Landing Outcome

Bryan Wai Kit Li
April 19, 2026

Executive Summary

This project explores the feasibility of predicting the landing outcome of Falcon 9's first stage using machine learning (ML). Data was collected from SpaceX API and Wikipedia using Python libraries "requests" for API calls and BeautifulSoup for web scraping. It was followed by data cleaning, data normalization, and feature engineering using Python. Exploratory data analysis (EDA) was conducted to identify ideal features for model training. Data visualization was done using Matplotlib, Folium, and Poly Dash. Summary statistics on the data was generated using SQLLite3. ML algorithms were trained and tested to identify the best one for prediction using Scikit-Learn (SL). Support Vector Machine (SVM) was identified as the best at the end.

Introduction

SpaceY, founded by Allon Mask, plans to enter into the space travel industry. It will put SpaceY in direct competition with SpaceX, founded by Elon Musk. One competitive advantage SpaceX has is cost. It could attempt landing its Falcon 9's first stage and, if successful, can reuse it. It thus could possibly bid lower for projects than its competitors, if it deems the first stage can be landed successfully and reused. To be competitive in the bidding process with SpaceX, SpaceY wants to predict what the final ultimate cost would be for SpaceX. It wants to predict the landing outcome with Falcon 9's first stage. It will then predict how SpaceX will bid. SpaceY wants to make such predictions using ML.

Methodology

Data for model development and evaluation would be retrieved from SpaceX API and Wikipedia. API calls and web scraping would be performed using Python libraries. Data wrangling including data cleaning, data normalization, and feature engineering would be performed. In data collection, a dataset of flight records would be retrieved from SpaceX API via REST API. Additionally, a dataset of flight records would be scraped from a Wikipedia webpage. Then the collected dataset of flight records would be pre-processed before use. It would entail such tasks as data cleaning, data normalization, and feature engineering.

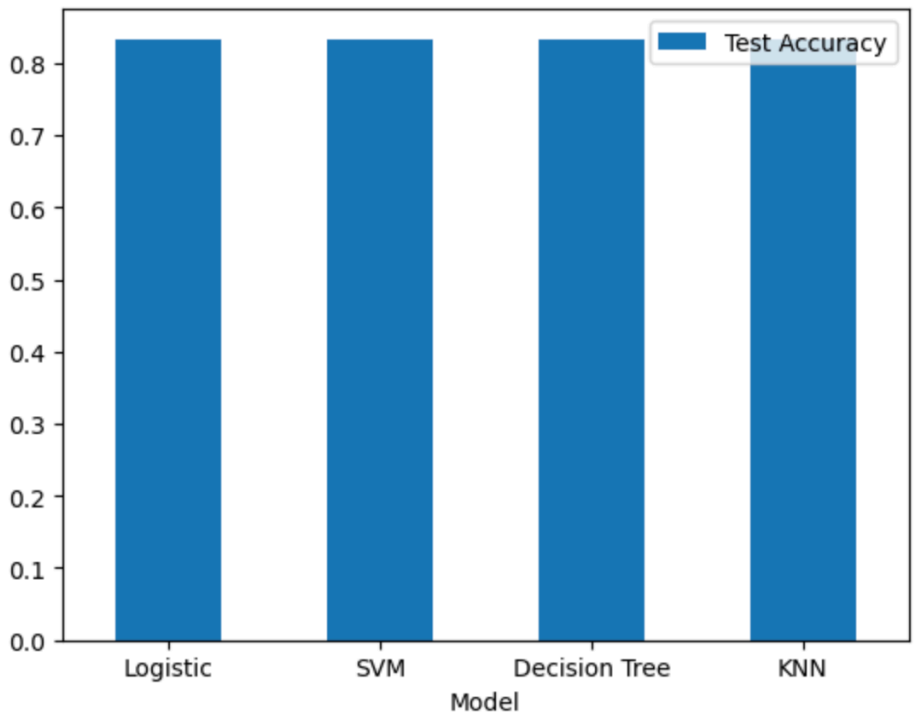
Then for EDA, charts would be created using Matplotlib to examine relationships among the features and target. Additionally, SQL queries would be run using SQLLite3 to examine flight records w. summary statistics. Folium would also be used to visualize launch sites geographically. Plotly Dash would be used to visualize landing outcomes by launch site or payload mass carried.

SL would be used in order to train and evaluate ML algorithms in order to identify the best estimator. Specifically, the dataset would be split into the training set and testing set for cross validation (CV). The training set would be further divided into the pure training set and validation set. Whereas the former would be used to fit the ML algorithm given a set of hyper parameters, the latter would be used to test such fitted ML

algorithm and calculate the “accuracy” score. Whichever combination of hyperparameters trained on had the best mean “accuracy” score across the CV iterations would be chosen for the ML algorithm’s best estimator. The generation of the best estimator would be complete by refitting the ML algorithm on the best hyperparameters to the whole training set. This process would be repeated for each of the ML algorithms evaluated. Whichever ML algorithm had the best “accuracy” score on the testing set would be chosen for prediction. The ML algorithms to consider include Logistic Regression (LR), SVM, Decision Tree (DT), and K-Nearest Neighbors (KNN).

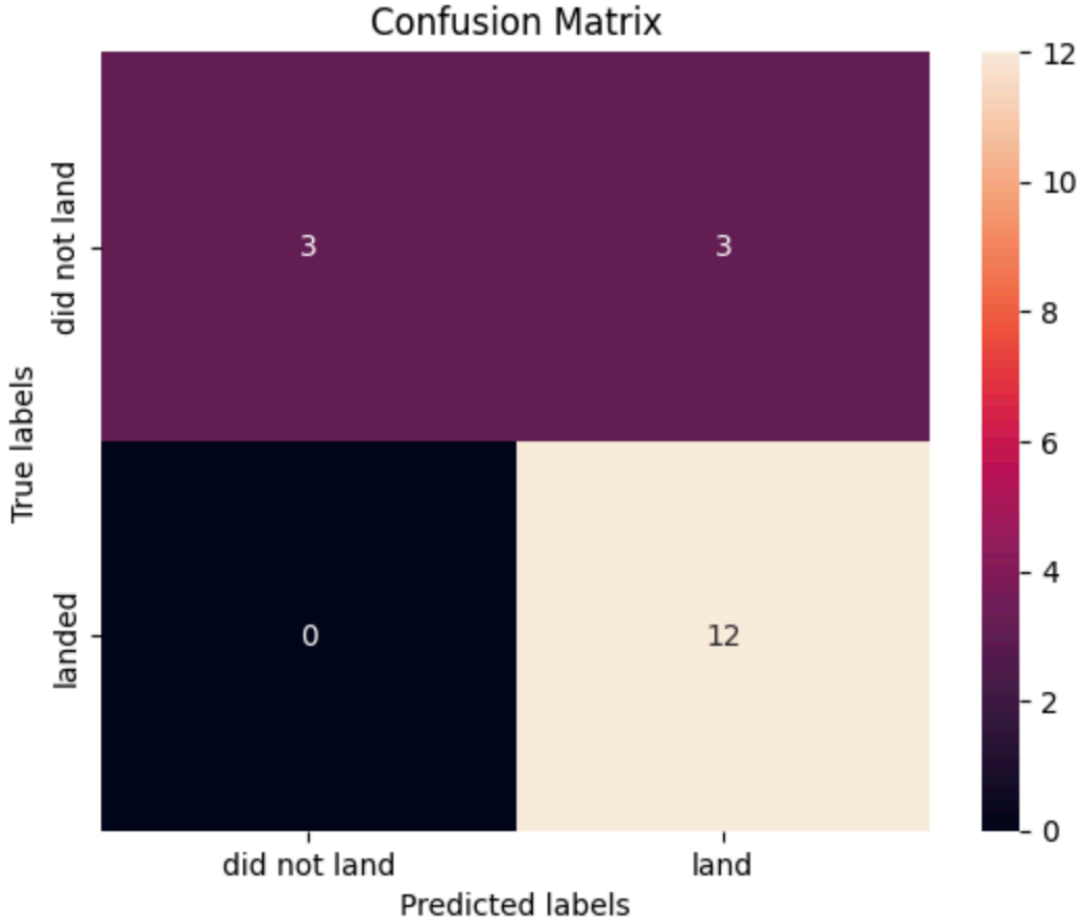
Results

“Accuracy” score on testing set:



- CV specifications
  - Sample size = 90; size of the training set = 72; size of the testing set = 18
  - # of folds for CV = 10
  - Targets: “FlightNumber”, “PayloadMass”, “Flights”, “GridFins”, “Reused”, “Legs”, “Block”, “ReusedCount”, “Orbit”, “LaunchSite”, “LandingPad”, and “Serial”
  - Each ML algorithm’s HP values to train on (see Appendix)
- CV results
  - Each ML algorithm’s best mean “accuracy” score (see below)
  - Each ML algorithm’s best HP values (see Appendix)
  - Each best estimator’s “accuracy” score on the testing set: see above

Confusion matrix on testing set:



As all four ML algorithms have the same confusion matrix, they do not only share the same “accuracy” score on the testing set, but also the same “precision”, “recall”, and “F1” scores. All four ML algorithms correctly predict all successful landing outcomes. However, they also have three false positives, which are of the error type costly here.

Other metrics:

ML Algorithm	Best Mean Accuracy	Test Accuracy	Roc Score
Logistic	0.846429	0.833333	0.888889
SVM	0.848214	0.833333	0.958333
Decision Tree	0.873214	0.833333	0.868056
KNN	0.848214	0.833333	0.895833

To distinguish the four ML algorithms, alternative measure ROC score is used, using SL's function "roc_auc_score". SVM has the highest ROC score, indicating that it best distinguishes classes moving across probability thresholds and covers the largest area under the ROC curve. Although DT has the best mean "accuracy" score, it does not generalize better than others based on its best estimator's "accuracy" score on the testing set. Although SVM appears to be the winner, the caveat is that the sample size of 90 is small.

Appendix

- Additional CV specifications
  - LR's HP values to train on = {"C":[0.01,0.1,1],'penalty':['l2'],'solver':['lbfgs']}
  - SVM's HP values to train on = {'kernel':('linear','rbf','poly','rbf', 'sigmoid'),'C':np.logspace(-3,3,5),'gamma':np.logspace(-3,3,5)}
  - DT's HP values to train on = {'criterion':['gini','entropy'],'splitter':['best','random'],'max_depth':[2*n for n in range(1,10)],'max_features':['log2','sqrt'],'min_samples_leaf':[1,2,4],'min_samples_split':[2,5,10]}
  - KNN's HP values to train on = {'n_neighbors':[1,2,3,4,5,6,7,8,9,10],'algorithm':['auto','ball_tree','kd_tree','brute'],'p':[1,2]}
- Additional CV results
  - LR's best HP values = {'C':0.01,'penalty':'l2','solver':'lbfgs'}
  - SVM's best HP values = {'C':1.0,'gamma':0.03162277660168379,'kernel':'sigmoid'}
  - DT's best HP values = {'criterion':'gini','max_depth':12,'max_features':'log2','min_samples_leaf':1,'min_samples_split':5,'splitter':'best'}
  - KNN's best HP values = {'algorithm':'auto','n_neighbors':10,'p':1}
- Project's Github link: https://github.com/liwaikitbryan/IBM-DS-2026-01-28-Capstone